

Real-Time Communication Cheat Sheet

Polling, Long Polling, WebSocket, SSE, Webhook: interview edition

1. Polling



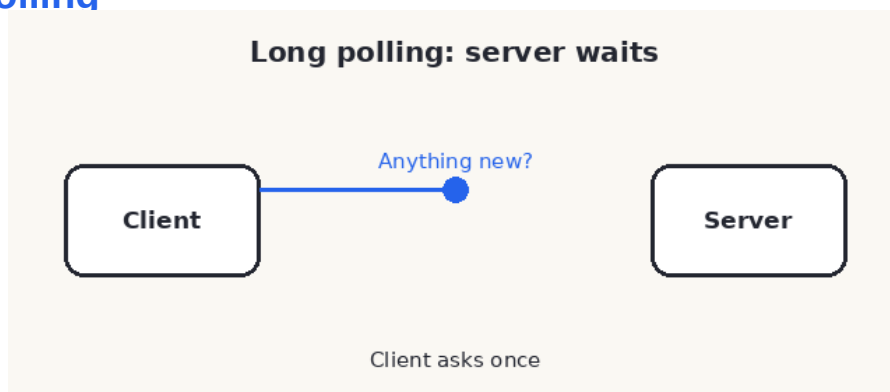
The client asks the server "anything new?" on a timer. Simple, but most answers are empty.

REMEMBER: "Are we there yet?" every 10 seconds. Timer is on the CLIENT.

```
async function poll() {
  const res = await fetch("/api/battery-status");
  console.log(await res.json());
  setTimeout(poll, 10000); // wait, then ask again
}
poll();
```

Interview tip: Use for slow updates (minutes). Too wasteful for every-second updates.

2. Long Polling



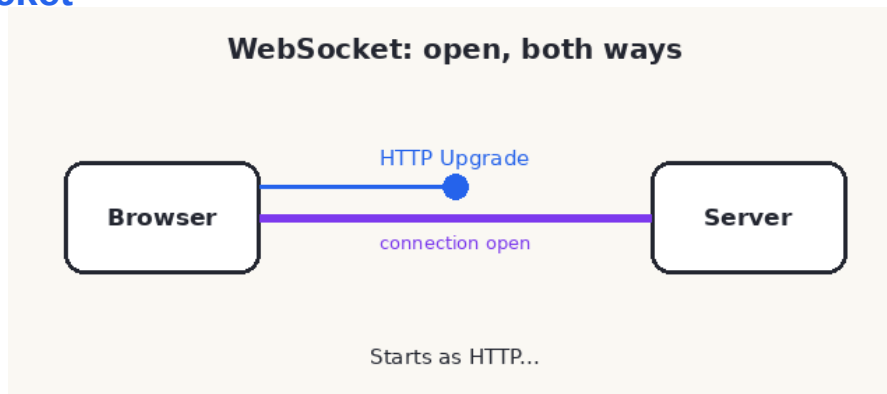
The client asks once; the server holds the request until data arrives (or ~30s timeout). The client asks again immediately.

REMEMBER: Normal polling = client waits. Long polling = SERVER waits.

```
// server (Express)
let waiting = [];
app.get("/api/updates", (req, res) => waiting.push(res));
function onNewData(data) {
  waiting.forEach(r => r.json({ data }));
  waiting = [];
}
```

Interview tip: Near real-time with plain HTTP. Good fallback when WebSocket is blocked.

3. WebSocket



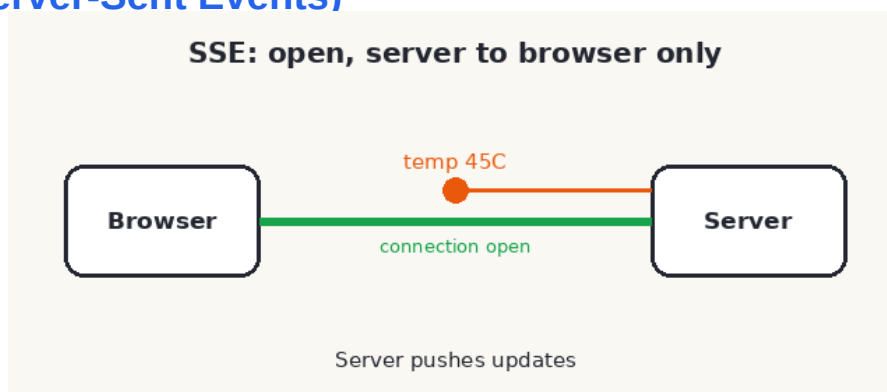
Starts as an HTTP request asking to "Upgrade". Server replies 101 Switching Protocols. After that, one open connection where BOTH sides send anytime.

REMEMBER: A phone call. Two-directional = either side can START talking.

```
const ws = new WebSocket("wss://app.com/live");
ws.onmessage = (m) => console.log("Server:", m.data);
ws.send("fast-charge ON"); // client talks anytime
```

Interview tip: Use for chat, games, live controls. Scale across servers with Redis Pub/Sub. Add ping/pong heartbeats and reconnect with backoff (1s, 2s, 4s...).

4. SSE (Server-Sent Events)



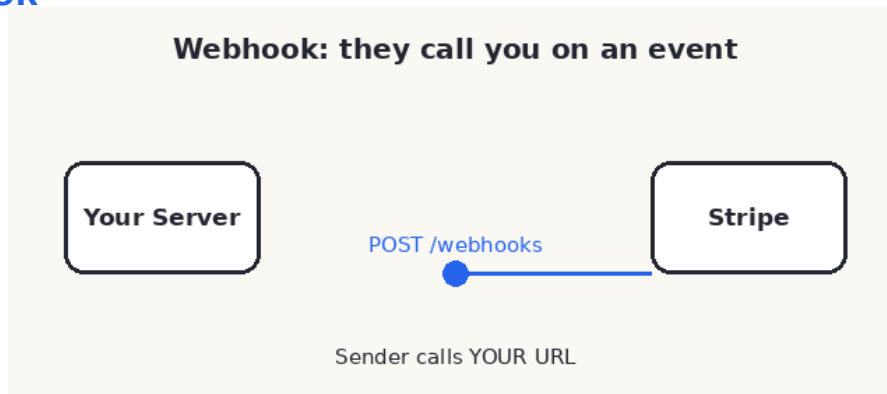
An open HTTP connection where only the server pushes to the browser. The browser reconnects automatically.

REMEMBER: A radio. You only listen. Best for live dashboards.

```
const events = new EventSource("/api/battery-live");
events.onmessage = (e) => console.log("New:", e.data);
```

Interview tip: Live battery temperature dashboard? SSE is simpler than WebSocket because the user only watches.

5. Webhook



Just a normal HTTP POST that ANOTHER server sends to YOUR URL when an event happens. A "reverse API".

REMEMBER: A text notification. They call you; you reply 200 fast.

```
app.post("/webhooks/payment", async (req, res) => {
  const e = req.body; // 1. verify signature first!
  if (await db.events.findOne({ id: e.id }))
    return res.status(200).send("dup"); // 2. idempotency
  await db.events.insert({ id: e.id });
  queue.add(e); // 3. heavy work in background
  res.status(200).send("OK");
});
```

Interview tip: Three must-say words: signature verification, idempotency (event ID), retries.

Decision Table

Need	Use
Slow updates (minutes)	Polling
Near real-time, plain HTTP only	Long polling
Live, server to browser only	SSE
Live, both directions (chat, games, controls)	WebSocket
Server-to-server event alerts	Webhook

Quick-Fire Interview Answers

Q: Is a webhook special technology?

A: No. It is a normal HTTP POST. The pattern (they call you on an event) makes it a webhook.

Q: How does a WebSocket start?

A: HTTP request with Upgrade header, server replies 101 Switching Protocols.

Q: Problems with always-open connections?

A: Server memory, silent drops (use ping/pong), reconnect logic, load balancer timeouts, reconnect storms on deploy.

Q: User A on Server 1, User B on Server 2?

A: Servers publish/subscribe through Redis Pub/Sub so any server can deliver to any user.

Q: Webhook arrives twice?

A: Retries happen. Store the event ID and skip duplicates (idempotency). Still return 200.

Q: Why reconnect with backoff?

A: Stops thousands of clients hammering the server at the same moment after a restart.